SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192319056**
Name	**THARIKA SHREE M**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I **Tharika Shree M (192319056)***(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tharika Shree M

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

1. Hospital Doctor Shift Assignment Using Backtracking Search (10 Marks)

Problem statement

A hospital must assign three doctors (D1, D2, D3) to three shifts (Morning, Afternoon, Night), satisfying all the given constraints.

Step 1: Define variables and domains

Variables:

- (D1): Shift assigned to Doctor 1
- (D2): Shift assigned to Doctor 2
- (D3): Shift assigned to Doctor 3

Initial domain for each doctor:

{M,A,N\}

Here, \(M\) = Morning, \(A\) = Afternoon, and \(N\) = Night.

Step 2: Identify constraints

1. D1 cannot work Night: \(D1 \ne N\).
2. D2 must work before D3: \(D2 < D3\), where \(M<A<N\).
3. Only one doctor can work each shift.
4. D3 cannot work Morning: \(D3 \ne M\).
5. Each doctor must be assigned exactly one shift.

Step 3: Apply backtracking search

Backtracking assigns a shift to one doctor at a time. If an assignment violates a constraint, the algorithm rejects it and tries another value.

Step	Assignment	Constraint check	Result
1	D1 = Morning	D1 is not assigned Night	Valid
2	D2 = Morning	Morning is already assigned to D1	Reject
3	D2 = Afternoon	D2 works after D1; shifts remain unique	Valid
4	D3 = Morning	D3 cannot work Morning; Morning is occupied	Reject
5	D3 = Afternoon	Afternoon is already assigned to D2	Reject
6	D3 = Night	D2 works Afternoon before D3 at Night; all shifts are unique	Valid

Step 4: Final valid schedule

Doctor	Assigned shift
D1	Morning
D2	Afternoon
D3	Night

The final assignment satisfies all five constraints.

Backtracking pseudocode

```
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
assignment = {}

def valid(doctor, shift):
```

```
    if shift in assignment.values():
        return False

    if doctor == "D1" and shift == "Night":
        return False

    if doctor == "D3" and shift == "Morning":
        return False

    temp = assignment.copy()
    temp[doctor] = shift

    if "D2" in temp and "D3" in temp:
        if shifts.index(temp["D2"]) >= shifts.index(temp["D3"]):
            return False

    return True

def backtrack(index):
    if index == len(doctors):
        return True

    doctor = doctors[index]

    for shift in shifts:
        if valid(doctor, shift):
            assignment[doctor] = shift

            if backtrack(index + 1):
                return True

            del assignment[doctor]

    return False

if backtrack(0):
    print(assignment)
else:
    print("No valid schedule")
```

Conclusion

Backtracking search systematically explores assignments and rejects those that violate the constraints. The valid schedule is D1–Morning, D2–Afternoon, D3–Night.

2. Robot Navigation in a 5×5 Grid Using BFS and Manhattan Distance (10 Marks)

Problem statement

A robot must travel from Start (S) to Goal (G) in a 5×5 grid. It can move Up, Down, Left, or Right. Obstacles cannot be crossed, and each movement has a cost of 1.

Step 1: Example grid

Since the original grid image is missing, consider the following illustrative grid. Coordinates are written as (row, column).

For this example:

Start

Goal

Obstacle (X)

For this example:

- Start: (1,1)
- Goal: (5,5)
- Obstacles: (1,3), (2,2), (2,3), (4,4)
- Movement cost: 1 per step

Step 2: Manhattan distance heuristic

The Manhattan distance estimates the number of horizontal and vertical moves required to reach the goal.

The heuristic is useful for estimating the remaining distance, but ordinary BFS does not use a heuristic or a priority queue. BFS uses a FIFO queue. If the question requires Manhattan distance to guide node selection, A* search is the appropriate algorithm.

Step 3: BFS algorithm

1. Insert the starting cell into a FIFO queue.
2. Mark the starting cell as visited.
3. Remove the front cell from the queue.
4. Check whether it is the goal.
5. Add each valid, unvisited neighbouring cell in the order Up, Down, Left, Right.
6. Store each cell's parent so that the path can be reconstructed.
7. Continue until the goal is found or the queue becomes empty.

Step 4: Node expansion and queue updates

For the example grid, the initial queue is:

$$Q = [(1,1)]$$

Using the neighbour order Up, Down, Left, Right:

Step	Expanded node	New valid nodes added	Queue after expansion
1	(1,1)	(2,1), (1,2)	[(2,1), (1,2)]
2	(2,1)	(3,1)	[(1,2), (3,1)]
3	(1,2)	None; remaining neighbours are blocked or visited	[(3,1)]
4	(3,1)	(4,1), (3,2)	[(4,1), (3,2)]
5	(4,1)	(5,1), (4,2)	[(3,2), (5,1), (4,2)]

6	(3,2)	(3,3)	[(5,1), (4,2), (3,3)]
7	(5,1)	(5,2)	[(4,2), (3,3), (5,2)]
8	(4,2)	(4,3)	[(3,3), (5,2), (4,3)]
9	(3,3)	(3,4)	[(5,2), (4,3), (3,4)]
10	(5,2)	(5,3)	[(4,3), (3,4), (5,3)]
11	(4,3)	None; next cells are blocked or visited	[(3,4), (5,3)]
12	(3,4)	(2,4), (3,5)	[(5,3), (2,4), (3,5)]
13	(5,3)	(5,4)	[(2,4), (3,5), (5,4)]
14	(2,4)	(1,4), (2,5)	[(3,5), (5,4), (1,4), (2,5)]
15	(3,5)	(4,5)	[(5,4), (1,4), (2,5), (4,5)]
16	(5,4)	(5,5)	[(1,4), (2,5), (4,5), (5,5)]
17	(5,5)	Goal reached	Stop

Step 5: Optimal path and cost

One shortest path found by BFS is:

$$(1,1) \rightarrow (2,1) \rightarrow (3,1)$$
$$\rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4)$$
$$\rightarrow (3,5) \rightarrow (4,5) \rightarrow (5,5)$$

Number of movements:

$$\text{Path cost} = 8 \times 1 = \boxed{8}$$

BFS guarantees the shortest path in terms of number of movements because all moves have equal cost.

Complexity

- Time complexity: \(O(V+E)\) for graph search, where \(V\) is the number of traversable cells and \(E\) is the number of valid connections.
- Space complexity: \(O(V)\) for the visited set, queue, and parent information.

Conclusion

BFS is suitable for an unweighted grid because it finds a shortest path while avoiding obstacles. Manhattan distance can estimate proximity to the goal; however, if the assessment specifically expects heuristic-based node ordering, A* should be used instead of ordinary BFS.

BFS table clarification: After Step 16, the goal is added to the queue but is not expanded immediately. Nodes already ahead of it must be expanded first. Continuing the queue correctly gives:

Step	Expanded node	Queue after expansion
17	(1,4)	(2,5), (4,5), (5,5), (1,5)
18	(2,5)	(4,5), (5,5), (1,5)

19	(4,5)	(5,5), (1,5)
20	(5,5)	Goal reached

The shortest path cost remains 8. Since the goal was first discovered from (5,4), the parent-based path reconstruction follows the bottom row.

3. Autonomous Rescue Robot in a Partially Observable Building (10 Marks)

a) Analyze the constraints and their effect on decision-making

The rescue robot must reach a survivor at goal position \(G\) while minimizing movement cost and maintaining safety.

Constraint	Effect on the robot
Movement in four directions	The robot can move Up, Down, Left, or Right.
Obstacles	Blocked rooms must be excluded from possible routes.
Limited sensing	Only adjacent cells can be observed, so the robot cannot know the complete building layout initially.
Movement cost = 1	Every ordinary move adds one unit to the total path cost.
Risky zones add cost 2	Entering a risky cell costs \(1+2=3\) units.
Minimum total cost	The robot must compare alternative routes based on accumulated cost.
Dynamic knowledge	The robot must update its map after observing new cells and replan when necessary.
Safe navigation	Routes involving unacceptable danger should be avoided even if they appear cheaper.

The robot must distinguish between unknown, free, blocked, and risky cells. An unknown cell should not automatically be treated as safe.

b) Solution using Uniform Cost Search and an Online Search Agent

Uniform Cost Search (UCS) is appropriate because the movement costs are not equal: ordinary cells cost 1, while risky cells cost 3 to enter.

The online search agent supports navigation when the full environment is unknown. It combines sensing, action, and map updates with cost-based planning.

Algorithm steps

1. Initialize the robot's current position as \(S\) with path cost zero.
2. Observe the four adjacent cells using its sensors.
3. Update the internal map with any newly discovered obstacles, safe cells, and risky zones.
4. Add feasible moves to a priority queue ordered by accumulated cost.
5. Select the reachable state with the smallest known cost.
6. Reject blocked cells and unsafe routes according to the robot's safety policy.
7. Move toward the selected cell and observe the environment again.

8. Update the map and costs when new information becomes available.
9. Replan if a newly discovered obstacle or risk changes the best route.
10. Stop when the survivor's location \(G\) is reached.

The transition cost is:

$$c(n, n') = \begin{cases} 1, & \text{if } n' \text{ is an ordinary cell} \\ 3, & \text{if } n' \text{ is a risky cell} \end{cases}$$

The accumulated cost is:

$$g(n') = g(n) + c(n, n')$$

UCS always expands the frontier node with the smallest accumulated cost. With non-negative costs and a sufficiently accurate map of the relevant routes, it finds a minimum-cost path.

Illustrative example

Suppose two possible routes reach the survivor:

Route	Ordinary moves	Risky cells	Total cost
Route A	6	1	(6+2=8)
Route B	8	0	(8+0=8)

Both routes cost 8 units. If the robot's safety policy prohibits entering the risky zone, it must choose Route B. If risky zones are allowed and both routes meet the safety requirements, additional factors such as exposure time can determine the preferred route.

This example illustrates how cost and safety constraints work together; the actual route cannot be calculated without the building's grid and obstacle locations.

c) Application of AI concepts

1. Intelligent agents

An intelligent agent perceives its environment through sensors and acts through motors or other actuators.

For the rescue robot:

- Sensors: Detect nearby walls, obstacles, survivors, and hazardous conditions.
- Actuators: Move the robot between rooms and operate rescue equipment.
- Internal model: Stores the known map, visited rooms, risks, and available paths.
- Decision-making: Selects the next movement based on the current position, observations, and path costs.

A model-based, goal-based agent with cost-sensitive planning is appropriate for this task.

2. Rationality

A rational agent selects actions that maximize its expected performance based on the information currently available.

The rescue robot should:

- Prefer low-cost routes that satisfy safety requirements.
- Avoid blocked rooms and unacceptable hazards.
- Gather information when uncertainty could affect route selection.
- Update its decisions when new obstacles or risks are discovered.
- Reach the survivor while minimizing unnecessary movement and danger.

Rationality does not mean that the robot always knows the perfect route. It means that its actions are appropriate given its current knowledge and performance objectives.

3. Environment type

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed.
Determinism	Uncertain in practice	Hidden obstacles and hazards may be discovered during movement.
Episodic or sequential	Sequential	Each action affects later decisions.
Static or dynamic	Potentially dynamic	Hazards and accessible routes may change during rescue operations.
Discrete or continuous	Primarily discrete	Rooms and movements are represented as grid cells and discrete actions.
Single-agent or multi-agent	Single-agent model	One robot is responsible for navigation, although human rescuers may assist.
Known or unknown	Initially unknown	The robot must learn the layout through observations.

d) Justification for the selected approach

A combination of a model-based online search agent and Uniform Cost Search is suitable for the following reasons:

1. Handles unequal costs: UCS considers the additional cost of entering risky cells rather than simply counting the number of movements.
2. Supports changing knowledge: The online agent updates its internal map as it discovers obstacles and hazards.
3. Improves decision-making: The robot can compare different routes and choose one that satisfies the safety requirements at the lowest cost.
4. Avoids unnecessary exploration: The internal map helps the robot remember previously visited locations and discovered obstacles.
5. Supports rational behaviour: Decisions are based on the current information, accumulated cost, and safety requirements.

One important limitation is that ordinary UCS assumes the relevant transition costs are known. In a partially observable building, the robot must first discover local conditions and repeatedly replan. Therefore, online exploration combined with UCS-based cost planning is more appropriate than running UCS once on a completely known map.

Conclusion

The rescue robot should use a model-based online search agent with Uniform Cost Search for cost-sensitive navigation. This approach combines sensing, internal map updates, rational decision-making, and minimum-cost planning. By incorporating a safety policy, the robot can navigate uncertain environments more effectively while minimizing unnecessary movement and exposure to danger.